

order Service

```
package jp.co.activekenshu.delivery.service;

import java.sql.Connection;
import java.sql.SQLException;

import jp.co.activekenshu.delivery.config.DbConfig;
import jp.co.activekenshu.delivery.dto.OrderDto;
import jp.co.activekenshu.delivery.repository.OrderRepository;
import jp.co.activekenshu.delivery.validator.OrderValidator;

/**
 * 配送依頼機能の業務処理を担当する Service クラスです。
 *
 * <p>
 * Service とは、処理の流れを管理するクラスです。
 * Servlet から呼ばれて、入力チェックや DB 登録処理を行います。
 * </p>
 *
 * <p>
 * SQL 自体は Repository に任せます。
 * Service は「どの順番で処理するか」を担当します。
 * </p>
 */
public class OrderService {

    /**
     * 配送依頼情報の DB 操作を行う Repository です。
     */
    private final OrderRepository orderRepository = new OrderRepository();

    /**
     * 配送依頼情報の入力チェックを行う Validator です。
     */
    private final OrderValidator orderValidator = new OrderValidator();

    /**
     * 配送依頼情報の入力チェックを行います。
     *
     * <p>
```

```

* Servletから確認画面に進む前にも使います。
* 登録前にも同じチェックを使います。
* </p>
*
* @param order 配送依頼情報
* @throws IllegalArgumentException 入力値に問題がある場合
*/
public void validate(OrderDto order) {

    // 実際のチェック処理はValidatorに任せます。
    orderValidator.validate(order);
}

/**
* 配送依頼情報をDBに登録します。
*
* <p>
* 以下の順番で処理します。
* </p>
*
* <ol>
* <li>入力チェック</li>
* <li>DB接続取得</li>
* <li>トランザクション開始</li>
* <li>配送ID採番</li>
* <li>delivery_orderへINSERT</li>
* <li>delivery_situationへINSERT</li>
* <li>コミット</li>
* </ol>
*
* @param order 配送依頼情報
* @return 登録された配送ID
* @throws SQLException DB処理に失敗した場合
* @throws IllegalArgumentException 入力値に問題がある場合
*/
public String register(OrderDto order) throws SQLException {

    // DB登録前に必ず入力チェックを行います。
    orderValidator.validate(order);

    // Connectionはfinallyでcloseしたいので、tryの外で宣言します。

```

```
Connection conn = null;

try {
    // DB 接続を取得します。
    conn = DbConfig.getConnection();

    // 自動コミットを OFF にします。
    // これにより、INSERT だけでは DB に確定されません。
    conn.setAutoCommit(false);

    // sequence を使って配送 ID を採番します。
    String deliveryId = orderRepository.getNextDeliveryId(conn);

    // delivery_order テーブルに配送依頼情報を登録します。
    orderRepository.insertOrder(conn, deliveryId, order);

    // delivery_situation テーブルに配送状況の初期値を登録します。
    orderRepository.insertSituation(conn, deliveryId);

    // ここまで全部成功したら、DB への変更を確定します。
    conn.commit();

    // 完了画面に表示するために配送 ID を返します。
    return deliveryId;

} catch (SQLException e) {

    // SQL エラーが起きた場合は、途中までの登録を取り消します。
    if (conn != null) {
        conn.rollback();
    }

    // Servlet 側にエラーを伝えるため、例外を投げ直します。
    throw e;

} finally {

    // DB 接続が開いている場合は必ず閉じます。
    if (conn != null) {
        conn.close();
    }
}
```

}

}

}